ResNet50

when I try to use the Cifar10 dataset with Resnet50, I am getting an output size of [232, 5568], but the imput spatial dimensions is the correct [3072]? Is there a reason for that?

Yes — and the key distinction is that **3072 is the number of raw input values, not the dimension ResNet-50 is required to preserve**.

For CIFAR-10, each image is:

$$3 \times 32 \times 32 = 3072$$

So before entering ResNet, a batch should normally have shape

```
[B, 3, 32, 32]
```

not `[B, 3072]`. ResNet is convolutional, so as the tensor moves through the network, **the spatial resolution shrinks while the number of channels grows**.

For a standard ResNet-50, the rough progression for a `32×32` CIFAR image is:

Stage	Shape
Input	`[B, 3, 32, 32]`
conv1, stride 2	`[B, 64, 16, 16]`
maxpool, stride 2	`[B, 64, 8, 8]`
layer1	`[B, 256, 8, 8]`
layer2	`[B, 512, 4, 4]`
layer3	`[B, 1024, 2, 2]`
layer4	`[B, 2048, 1, 1]`

Stage	Shape
avgpool + flatten	`[B, 2048]`
classifier	`[B, 10]` if modified for CIFAR-10

So `[232, 5568]` **is not a normal final output shape for an unmodified ResNet-50**. The `232` very plausibly looks like your **batch size**, while `5568` looks like you've flattened or concatenated some intermediate representation.

The fastest way to locate exactly where that dimension appears is to print the shapes through the forward pass:

```
print("input:", x.shape)

x = model.conv1(x)
print("conv1:", x.shape)

x = model.bn1(x)
x = model.relu(x)
x = model.maxpool(x)
print("maxpool:", x.shape)

x = model.layer1(x)
print("layer1:", x.shape)

x = model.layer2(x)
print("layer2:", x.shape)

x = model.layer3(x)
print("layer3:", x.shape)

x = model.layer4(x)
print("layer4:", x.shape)

x = model.avgpool(x)
print("avgpool:", x.shape)

x = torch.flatten(x, 1)
print("flatten:", x.shape)
```

```
x = model.fc(x)
print("fc:", x.shape)
```

One particularly important CIFAR-10 detail

Standard ResNet-50 was designed for ImageNet's much larger `224×224` images. For CIFAR-10's tiny `32×32` images, people commonly replace the initial `7×7, stride=2` convolution and remove the max-pool:

```
model.conv1 = nn.Conv2d(
    3, 64,
    kernel_size=3,
    stride=1,
    padding=1,
    bias=False
)

model.maxpool = nn.Identity()
model.fc = nn.Linear(2048, 10)
```

That preserves spatial information much better for CIFAR-10.

Also, if you are seeing:

```
torch.Size([232, 5568])
```

show me **the few lines of code immediately before that shape is printed**, especially any `view()`, `reshape()`, `flatten()`, `cat()`, or forward-hook code. `5568` is unusual enough that we can probably pinpoint exactly where it comes from.

gpt-5-6-thinking

Well it's saying that the input and output must have the same number of spatial dimensions, and errors out.

That error strongly suggests the real problem is **tensor shape**, not ResNet-50 itself.

PyTorch's resize/interpolation operations expect image tensors to retain explicit spatial dimensions. For CIFAR-10, a batch should look like:

```
[B, 3, 32, 32]
```

So with your batch size of 232:

```
[232, 3, 32, 32]
```

If PyTorch is instead seeing:

```
[232, 5568]
```

then it sees a **2-D tensor**: batch dimension + feature dimension. There are no explicit height/width dimensions anymore.

For example, this will fail:

```
x.shape
# [232, 5568]

F.interpolate(x, size=(224, 224))
```

because `F.interpolate` interprets dimensions after `[N, C]` as spatial dimensions. Here there are **zero spatial dimensions**, but you've requested a 2-D output size `(224,224)`.

The error usually looks approximately like:

```
Input and output must have the same number of spatial dimensions,
but got input with spatial dimensions of [] and output size of (224, 224)
```

Check this first

Immediately before you pass the images into the model, print:

```
print(images.shape)
```

You want:

```
torch.Size([232, 3, 32, 32])
```

If you get:

```
torch.Size([232, 3072])
```

then you've flattened CIFAR-10:

```
3 * 32 * 32 = 3072
```

and that's the issue. **Do not flatten before ResNet.**

For example, this is wrong for a CNN:

```
images = images.view(images.size(0), -1)
outputs = model(images)
```

That converts

```
[232, 3, 32, 32]
```

into

```
[232, 3072]
```

Instead:

```
images = images.to(device)

print(images.shape)
# should be [232, 3, 32, 32]

outputs = model(images)
```

If you're resizing CIFAR-10 for ResNet-50

Do the resize while the image still has spatial dimensions:

```
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
])
```

Then your DataLoader should produce:

```
[232, 3, 224, 224]
```

Alternatively:

```
images = F.interpolate(
    images,
    size=(224, 224),
    mode="bilinear",
    align_corners=False
)
```

but `images` must already be:

```
[B, C, H, W]
```

not `[B, features]`.

The `[232, 5568]` value is especially suspicious. `5568` isn't a normal CIFAR-10 image size or normal ResNet-50 output dimension, so something in your pipeline is almost certainly **flattening, reshaping, or concatenating the tensor before the resize/model operation**.

If you paste the exact error traceback plus the code around your `Dataset` / `DataLoader`, transforms, and `model(images)` call, I can identify the exact offending line.

gpt-5-6-thinking

well my batchsize is 128, my input shape is 3072, and my output shape is 10.

That clears it up. The issue is that `3072` **is not a spatial shape**. It is a flattened feature count:

$$3072 = 3 \times 32 \times 32$$

For CIFAR-10, ResNet-50 expects each image to still have its channel/height/width structure. With batch size 128, the input should be:

```
[128, 3, 32, 32]
```

not:

```
[128, 3072]
```

Your desired output for CIFAR-10 is indeed:

```
[128, 10]
```

because there are 10 classes.

So somewhere before the ResNet, you're probably doing something like:

```
x = x.view(x.size(0), -1)
```

or:

```
x = torch.flatten(x, 1)
```

That is appropriate for a fully connected network, but **not before a convolutional network like ResNet**.

The correct data flow is:

```
[128, 3, 32, 32]
        ↓
     ResNet50
        ↓
    [128, 10]
```

If your dataset currently returns `[3072]` per image, reshape it before passing it to the network:

```
x = x.view(-1, 3, 32, 32)
```

Then:

```
outputs = model(x)
print(outputs.shape)
# torch.Size([128, 10])
```

Also make sure the final ResNet layer is changed from ImageNet's 1000 classes to CIFAR-10's 10:

```
model.fc = nn.Linear(model.fc.in_features, 10)
```

For CIFAR-10, I would also modify the first ResNet layer because the stock ResNet-50 aggressively downsamples tiny `32×32` images:

```
model.conv1 = nn.Conv2d(
    3,
    64,
    kernel_size=3,
    stride=1,
    padding=1,
    bias=False
)

model.maxpool = nn.Identity()
model.fc = nn.Linear(model.fc.in_features, 10)
```

So the fundamental distinction is:

```
3072              = number of values in one image
3 × 32 × 32       = actual image/spatial structure
```

ResNet needs the **second representation**, because its convolution layers operate over height and width.

gpt-5-6-thinking

how do i load resnet50 with random initial weights vs pretrained?

With `torchvision`, it's just the `weights` argument.

```
from torchvision.models import resnet50, ResNet50_Weights

# Random initialization
model_random = resnet50(weights=None)

# Pretrained on ImageNet
model_pretrained = resnet50(weights=ResNet50_Weights.DEFAULT)
```

For CIFAR-10, you'll also want to replace the final classifier:

```
import torch.nn as nn

model = resnet50(weights=None)   # or ResNet50_Weights.DEFAULT
model.fc = nn.Linear(model.fc.in_features, 10)
```

If you use pretrained weights, note that the original `fc` layer is pretrained for 1000 ImageNet classes, so replacing it with a 10-class layer means **that new final layer is randomly initialized**, while the rest of the network keeps its pretrained weights.

For CIFAR-10 specifically, a common setup is:

```
from torchvision.models import resnet50, ResNet50_Weights
import torch.nn as nn

model = resnet50(weights=ResNet50_Weights.DEFAULT)

model.conv1 = nn.Conv2d(
    3, 64,
    kernel_size=3,
    stride=1,
    padding=1,
    bias=False
)
```

```
model.maxpool = nn.Identity()
model.fc = nn.Linear(model.fc.in_features, 10)
```

One subtlety: if you replace `conv1` like that, you are also throwing away the pretrained first convolution. So for a cleaner **pretrained ResNet-50 baseline**, you may want to keep the original `conv1` and resize CIFAR-10 images to `224×224`. For a **CIFAR-style architecture trained from scratch**, `weights=None` plus the `3×3` conv modification is usually the more internally consistent setup.

gpt-5-6-thinking

what is the purpose of gelu vs relu? why would I want to use gelu within this kind of architecture?

Yes — GELU and ReLU serve the same broad purpose: they introduce **nonlinearity**. The interesting difference is *how* they decide what information gets through.

For your ResNet-50/CIFAR-10 setup, I would generally start with **ReLU**, because ResNet was designed and tuned around it. GELU is worth experimenting with, but it isn't automatically better.

ReLU: hard gating

ReLU is

$$\mathrm{ReLU}(x) = \max(0, x)$$

so:

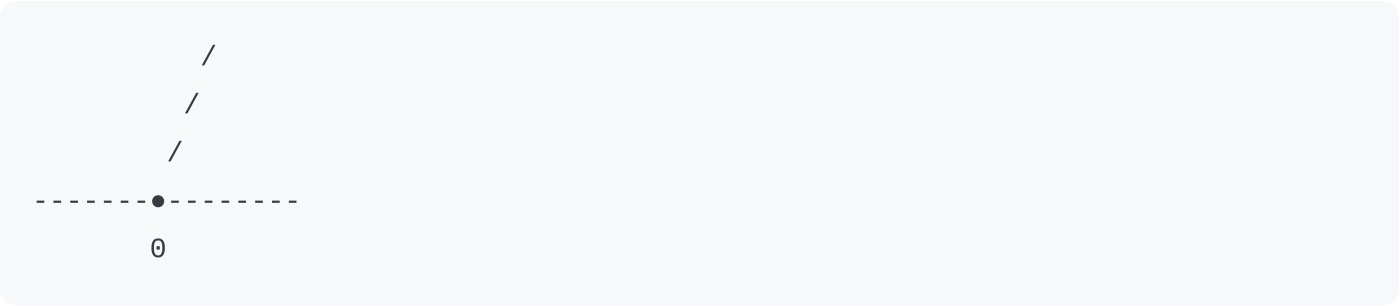
$$x = -2 \to 0, \qquad x = -0.1 \to 0, \qquad x = 0.1 \to 0.1, \qquad x = 2 \to 2$$

Conceptually:

```
negative activation ──> completely discarded
positive activation ──> passed through unchanged
```

This makes ReLU cheap and effective.

Its graph has a sharp corner at zero:



The derivative is approximately

$$\frac{d}{dx}\,\mathrm{ReLU}(x) = \begin{cases} 0 & x < 0 \\ 1 & x > 0 \end{cases}$$

That means negative activations get **zero gradient through the ReLU itself**.

This is where the "dying ReLU" phenomenon can come from: if a unit keeps landing in the negative region, it may stop receiving useful gradients.

GELU: soft gating

GELU stands for **Gaussian Error Linear Unit**.

Approximately,

$$\mathrm{GELU}(x) \approx \frac{x}{2}\left(1 + \tanh\left[\sqrt{\frac{2}{\pi}}\left(x + 0.044715x^3\right)\right]\right)$$

The exact expression is:

$$\mathrm{GELU}(x) = x\Phi(x)$$

where $\Phi(x)$ is the CDF of a standard Gaussian.

That formula sounds bizarre until you interpret it as:

> ***Instead of hard-thresholding an activation based only on whether it is positive, GELU smoothly weights the activation according to its magnitude.***

For example, approximately:

x	ReLU	GELU
-2	0	-0.046
-1	0	-0.159
-0.5	0	-0.154
0	0	0
0.5	0.5	0.346
1	1	0.841
2	2	1.954

Notice something neat: GELU lets **small negative values survive**.

So instead of:

```
ReLU:

negative ————————> kill it
positive ————————> keep it
```

GELU behaves more like:

```
very negative ——> mostly suppress
slightly negative -> partially preserve
near zero ————————> smoothly gate
positive ————————> increasingly preserve
large positive ——> almost unchanged
```

That's a significantly softer decision boundary.

Why might that help a neural network?

Suppose a convolution produces:

```
[-1.2, -0.2, 0.05, 0.4, 2.1]
```

ReLU turns that into:

```
[0, 0, 0.05, 0.4, 2.1]
```

Information about `-1.2` versus `-0.2` is completely lost.

GELU might produce roughly:

```
[-0.14, -0.08, 0.026, 0.26, 2.06]
```

Now the network can still distinguish those negative responses.

This can give optimization a smoother landscape because gradients do not abruptly jump from zero to one at $x = 0$.

Why GELU is so common in Transformers

You've probably seen:

```
nn.GELU()
```

constantly in Transformer architectures.

That's not accidental.

Transformers tend to have blocks like:

```
LayerNorm
   ↓
Linear
   ↓
GELU
   ↓
Linear
```

For example:

```
x = linear1(x)
x = gelu(x)
x = linear2(x)
```

GELU's smooth activation works extremely well in these large dense representations. BERT, GPT-like architectures, Vision Transformers, etc. commonly use GELU or related

smooth activations.

But you're using ResNet

This changes the recommendation.

Classic ResNet was designed around:

```
Conv
 ↓
BatchNorm
 ↓
ReLU
 ↓
Conv
 ↓
BatchNorm
 ↓
+
residual
 ↓
ReLU
```

The activation statistics, BatchNorm behavior, initialization, and architecture were all developed with ReLU in mind.

So a normal ResNet-50 contains things like:

```
self.relu = nn.ReLU(inplace=True)
```

Switching that to:

```
self.relu = nn.GELU()
```

is perfectly valid experimentally, but it creates a **different architecture**.

If you're training from scratch

This is actually a nice experiment.

You could compare:

```
model_relu = resnet50(weights=None)
```

against a version where all ReLUs are replaced by GELUs.

For example:

```
import torch.nn as nn

def replace_relu_with_gelu(module):
    for name, child in module.named_children():
        if isinstance(child, nn.ReLU):
            setattr(module, name, nn.GELU())
        else:
            replace_relu_with_gelu(child)

model = resnet50(weights=None)
replace_relu_with_gelu(model)
```

Then you'd have:

```
ResNet-50 + ReLU
vs
ResNet-50 + GELU
```

That is a scientifically meaningful comparison.

I'd track:

- training loss
- validation loss
- training accuracy
- validation accuracy
- convergence speed
- variance between runs

because GELU might improve one metric without meaningfully improving final generalization.

One major warning with pretrained ResNet

Suppose you do:

```
model = resnet50(
    weights=ResNet50_Weights.DEFAULT
)
```

Those weights were trained assuming the activation function is **ReLU**.

If you then replace every ReLU with GELU:

```
replace_relu_with_gelu(model)
```

you have changed the function computed by the pretrained network.

A learned convolution might have been relying on:

$$x < 0 \Rightarrow 0$$

but now it gets:

$$x < 0 \Rightarrow \text{small negative output}$$

So your pretrained filters are no longer operating under exactly the conditions they were trained for.

Fine-tuning can potentially adapt, but you shouldn't think of this as a free upgrade.

There is also an interesting ResNet-specific issue

Residual networks already help tremendously with gradient propagation.

A residual block looks conceptually like:

$$y = F(x) + x$$

so gradients have a direct route through the identity connection:

$$\frac{\partial y}{\partial x} = \frac{\partial F(x)}{\partial x} + 1$$

That **+1** is enormously important.

Even if the nonlinear branch has poor gradients, the residual connection provides another gradient path.

So one of GELU's advantages—smoother gradient propagation—is arguably less critical in ResNet than it might be in architectures without skip connections.

ReLU vs GELU for your experiment

For your CIFAR-10 ResNet-50 project I'd think of it this way:

Property	ReLU	GELU
Negative values	Set to zero	Smoothly suppressed
Smooth	No	Yes
Computation	Very cheap	More expensive
Sparse activations	Yes	Much less
Standard for classic ResNet	**Yes**	No
Standard for Transformers	Rare	**Yes**
Compatible with pretrained ResNet behavior	**Yes**	Not exactly
Worth testing from scratch	Yes	**Yes**

So if your experiment is something like **pretrained vs random initialization**, I would *not* simultaneously change ReLU → GELU.

You want controlled experiments:

```
Experiment 1
Random weights + ReLU

Experiment 2
Pretrained weights + ReLU
```

Then you're measuring:

$$\boxed{\text{effect of pretraining}}$$

If you instead do:

```
Random + GELU

vs

Pretrained + ReLU
```

and one wins, you don't know whether the result came from:

- initialization,
- pretraining,
- activation function,
- interaction between activation and pretraining.

That's a confounded experiment.

Then, separately, run:

```
Experiment 3
Random weights + ReLU

Experiment 4
Random weights + GELU
```

Now you're isolating:

effect of activation function

And *that* would actually be a pretty interesting CIFAR-10 experiment, because you could examine whether GELU's smooth gating helps optimization despite classic ResNet being designed around ReLU.

gpt-5-6-thinking

I need a custom architecture for the assignment, and I'm thinking about changing the input to be the normal cifar10/100 32x32, how might that affect the training?

Yes — for CIFAR-10/100, changing the architecture so it natively expects **32×32 inputs** is usually much more sensible than forcing the images into an ImageNet-style pipeline.

The biggest issue with a stock ResNet-50 is the **input stem**. It was designed for `224×224` images:

```
conv1 = nn.Conv2d(
    3, 64,
    kernel_size=7,
    stride=2,
    padding=3
)

maxpool = nn.MaxPool2d(
    kernel_size=3,
    stride=2,
    padding=1
)
```

That immediately reduces spatial resolution:

$$32 \times 32 \to 16 \times 16 \to 8 \times 8$$

So before you've even entered the first main residual stage, you've thrown away a huge amount of spatial detail.

For ImageNet:

$$224 \to 112 \to 56$$

which is fine.

For CIFAR:

$$32 \to 16 \to 8$$

which is extremely aggressive.

A CIFAR-style stem

A much better starting point is:

```
self.conv1 = nn.Conv2d(
    3,
    64,
    kernel_size=3,
    stride=1,
    padding=1,
    bias=False
)

self.bn1 = nn.BatchNorm2d(64)
self.gelu = nn.GELU()

# no maxpool
```

Now your spatial progression starts like:

```
Input       32 × 32
Conv        32 × 32
Stage 1     32 × 32
Stage 2     16 × 16
Stage 3      8 × 8
Stage 4      4 × 4
AvgPool      1 × 1
```

That is much healthier for CIFAR.

Why this can improve training

The network gets to analyze edges, textures, and small structures **before downsampling them away**.

Consider a tiny feature that's only 2–3 pixels wide in the original CIFAR image. A stride-2 convolution followed by stride-2 max pooling can essentially destroy it immediately.

With:

```
3×3 conv, stride 1
```

the network can first build meaningful low-level features at full resolution.

That's particularly relevant for CIFAR because everything is tiny:

```
      32 pixels
   ┌──────────────┐
   │     🐱       │
   │              │
   │  entire cat  │
   │ fits in here │
   └──────────────┘
      32 pixels
```

You're asking the model to distinguish things like:

```
cat
dog
deer
horse
```

from a very small number of pixels. Early resolution preservation matters.

The tradeoff: more computation

There is a cost.

Convolutional compute is roughly proportional to

$$H \times W \times C_{in} \times C_{out} \times K^2$$

If you preserve `32×32` instead of immediately shrinking to `8×8`, early residual blocks operate on many more pixels.

Compare:

$$32^2 = 1024$$

versus

$$8^2 = 64$$

That's **16× as many spatial positions**.

So removing early downsampling increases:

- GPU memory consumption

- training time
- activation storage
- FLOPs

But CIFAR images are so small that this is usually quite manageable.

I'd also question whether you need a full ResNet-50

This is where your assignment could become interesting.

A normal ResNet-50 uses bottleneck blocks:

$$1 \times 1 \to 3 \times 3 \to 1 \times 1$$

and its stage widths eventually become:

```
256
512
1024
2048
```

That's a **huge network relative to 32×32 images**.

You can absolutely train it, but CIFAR-10 in particular may not require that much capacity.

A custom model could instead use something like:

```
Input: 3×32×32

3×3 Conv, 64
GELU
      ↓
32×32

Residual Stage
64 channels
      ↓
32×32

Residual Stage
```

```
128 channels
stride 2
      ↓
16×16

Residual Stage
256 channels
stride 2
      ↓
8×8

Residual Stage
512 channels
stride 2
      ↓
4×4

AdaptiveAvgPool
      ↓
512

Linear
      ↓
10 or 100
```

That is already a serious CNN.

You might implement a residual block like:

```python
class Block(nn.Module):
    def __init__(self, in_channels, out_channels, stride=1):
        super().__init__()

        self.conv1 = nn.Conv2d(
            in_channels,
            out_channels,
            kernel_size=3,
            stride=stride,
            padding=1,
            bias=False
        )
        self.bn1 = nn.BatchNorm2d(out_channels)
```

```
        self.act = nn.GELU()

        self.conv2 = nn.Conv2d(
            out_channels,
            out_channels,
            kernel_size=3,
            stride=1,
            padding=1,
            bias=False
        )
        self.bn2 = nn.BatchNorm2d(out_channels)

        if stride != 1 or in_channels != out_channels:
            self.skip = nn.Sequential(
                nn.Conv2d(
                    in_channels,
                    out_channels,
                    kernel_size=1,
                    stride=stride,
                    bias=False
                ),
                nn.BatchNorm2d(out_channels)
            )
        else:
            self.skip = nn.Identity()

    def forward(self, x):
        identity = self.skip(x)

        x = self.conv1(x)
        x = self.bn1(x)
        x = self.act(x)

        x = self.conv2(x)
        x = self.bn2(x)

        x = x + identity
        x = self.act(x)

        return x
```

Now you're actually making meaningful architectural decisions rather than just renaming ResNet-50.

A nice architecture for the assignment

I'd consider something around:

```
class CIFARNet(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()

        self.stem = nn.Sequential(
            nn.Conv2d(3, 64, 3, padding=1, bias=False),
            nn.BatchNorm2d(64),
            nn.GELU()
        )

        self.stage1 = nn.Sequential(
            Block(64, 64),
            Block(64, 64)
        )

        self.stage2 = nn.Sequential(
            Block(64, 128, stride=2),
            Block(128, 128)
        )

        self.stage3 = nn.Sequential(
            Block(128, 256, stride=2),
            Block(256, 256)
        )

        self.stage4 = nn.Sequential(
            Block(256, 512, stride=2),
            Block(512, 512)
        )

        self.pool = nn.AdaptiveAvgPool2d((1, 1))
        self.fc = nn.Linear(512, num_classes)

    def forward(self, x):
        x = self.stem(x)
```

```
        x = self.stage1(x)
        x = self.stage2(x)
        x = self.stage3(x)
        x = self.stage4(x)

        x = self.pool(x)
        x = torch.flatten(x, 1)

        return self.fc(x)
```

Your dimensions become beautifully predictable:

```
[128,   3, 32, 32]
          ↓
[128,  64, 32, 32]
          ↓
[128,  64, 32, 32]
          ↓
[128, 128, 16, 16]
          ↓
[128, 256,  8,  8]
          ↓
[128, 512,  4,  4]
          ↓
[128, 512,  1,  1]
          ↓
[128, 512]
          ↓
[128, 10]
```

For CIFAR-100 you'd simply use:

```
model = CIFARNet(num_classes=100)
```

instead.

What I'd expect experimentally

Compared with a stock ImageNet-style ResNet stem, your CIFAR-native architecture would likely show **better early feature preservation and potentially faster**

useful learning, because it's not discarding 15/16 of the spatial positions before the main network really starts.

The more interesting design space for your assignment is probably:

$$\boxed{32\times32\text{-native stem} + \text{residual blocks} + \text{GELU} + \text{progressive channel expansion}}$$

rather than simply taking ResNet-50 unchanged.

And that gives you a strong architectural justification in a report: **you reduced early downsampling because CIFAR images are 32×32 rather than 224×224, preserving fine-scale spatial information during the early feature-extraction stages.**

gpt-5-6-thinking

What about dropout? How exactly would that better performance?

Dropout can improve **generalization**, not necessarily training accuracy. In fact, if it is helping, your training accuracy may get slightly worse while validation/test accuracy gets better.

The mechanism is simple but surprisingly deep.

During training, dropout randomly zeros activations:

$$y_i = \begin{cases} 0 & \text{with probability } p \\ \frac{x_i}{1-p} & \text{otherwise} \end{cases}$$

So with:

```
nn.Dropout(p=0.3)
```

about 30% of activations are removed on each forward pass.

That forces the network to avoid depending too heavily on any one feature.

For example, without dropout, a classifier might learn something fragile like:

```
"if feature 127 is strongly positive, predict cat"
```

With dropout, feature 127 periodically disappears during training. The model is pressured toward something more redundant:

```
"features 31, 84, 127, and 201 collectively indicate cat"
```

That tends to make the learned representation more robust.

Why this can improve CIFAR performance

Your custom CNN probably has enough capacity to memorize aspects of the training set.

You might see something like:

```
Training accuracy:   99%
Validation accuracy: 82%
```

That's classic overfitting.

Dropout can reduce that gap:

```
Training accuracy:   95%
Validation accuracy: 86%
```

So "better performance" usually means **better validation/test performance**, not maximizing the training-set score.

The underlying idea is that dropout injects noise into the model during training. Every minibatch effectively sees a slightly different network:

```
Batch 1:
A → B → X → D → E

Batch 2:
A → X → C → D → X

Batch 3:
X → B → C → X → E
```

where `x` means that activation was dropped.

You can loosely think of this as training a huge ensemble of overlapping subnetworks, then using the full network at inference time.

Where should you put dropout in your CNN?

This matters a lot.

For the architecture we were discussing:

```
Conv
BN
GELU
↓
Residual stages
↓
AdaptiveAvgPool
↓
Linear
```

the safest place to start is **right before the final classifier**:

```
self.pool = nn.AdaptiveAvgPool2d((1, 1))
self.dropout = nn.Dropout(p=0.3)
self.fc = nn.Linear(512, num_classes)
```

and:

```
x = self.pool(x)
x = torch.flatten(x, 1)

x = self.dropout(x)
x = self.fc(x)
```

So:

```
[batch, 512]
     ↓
Dropout(0.3)
     ↓
Linear
```

```
      ↓
[batch, 10]
```

That's a very reasonable first experiment.

Why not put dropout everywhere?

Because convolutional residual networks already have several regularizing effects:

- BatchNorm
- data augmentation
- weight decay
- residual connections
- stochastic minibatch training

Heavy dropout inside every residual block can actually make optimization worse.

This is especially true if you do something like:

```
Conv
↓
Dropout
↓
BatchNorm
```

because dropout changes the activation distribution while BatchNorm is trying to estimate it.

That interaction isn't necessarily catastrophic, but it makes the network's statistics noisier.

If you want dropout inside a block, something like this is more reasonable:

```
x = self.conv1(x)
x = self.bn1(x)
x = self.gelu(x)

x = self.dropout(x)
```

```
x = self.conv2(x)
x = self.bn2(x)
```

but I'd start without it.

There is also `Dropout2d`

This gets particularly interesting for CNNs.

Normal dropout:

```
nn.Dropout(0.2)
```

can independently remove individual activation values.

If you have:

```
[batch, 64, 32, 32]
```

it randomly knocks out individual elements.

`Dropout2d`, however:

```
nn.Dropout2d(0.1)
```

can remove entire feature maps/channels.

Conceptually:

```
Channel 0: keep
Channel 1: keep
Channel 2: DROP ENTIRE CHANNEL
Channel 3: keep
Channel 4: DROP ENTIRE CHANNEL
```

That can make more sense for CNNs because neighboring pixels within a feature map are highly correlated.

If ordinary dropout removes:

```
▮ ▮ ▮ X ▮ ▮
▮ X ▮ ▮ ▮ ▮
```

```
▉ ▉ ▉ ▉ x ▉
```

the surrounding activations can often compensate trivially.

Dropping a whole channel is a much stronger statement:

```
feature detector #17 is unavailable this iteration
```

Now the network has to learn alternative feature representations.

Residual networks have an even better alternative

For ResNet-like architectures, there is a related technique called **stochastic depth** or **DropPath**.

Instead of dropping individual neurons, you sometimes drop an entire residual branch.

Remember:

$$y = x + F(x)$$

Normally:

```
            ┌── F(x) ──┐
x ──────────┤          + → y
            └──────────┘
```

With stochastic depth, during some training steps:

```
            ┌── DROPPED ──┐
x ──────────┤             + → y
            └─────────────┘
```

so effectively:

$$y = x$$

for that block.

That's a very natural regularizer for residual architectures because the skip connection still keeps information flowing.

Modern residual-style models often benefit more from this than from aggressive conventional dropout inside convolutional blocks.

What I would do for your assignment

I would make the regularization progression controlled:

```
Baseline
CIFAR-native residual network
+ GELU
+ BatchNorm

Experiment A
+ Dropout(0.2) before classifier

Experiment B
+ Dropout(0.4) before classifier
```

Then compare:

Model	Train Acc	Val Acc	Val Loss
No dropout	99%	84%	0.62
p=0.2	97%	86%	0.51
p=0.4	93%	84%	0.58

Those numbers are hypothetical, but that pattern illustrates what you're looking for.

If `p=0.2` lowers training accuracy slightly **while raising validation accuracy**, dropout is doing exactly what you want.

If both training and validation accuracy collapse, you're regularizing too strongly.

For CIFAR-10/100, I'd start here:

```
self.dropout = nn.Dropout(0.2)
```

immediately before the final linear layer.

Then I'd treat `0.0`, `0.2`, and `0.4` as an ablation study.

For your architecture specifically, **dropout before the classifier + weight decay + CIFAR data augmentation** is probably a cleaner design than scattering dropout through every residual block. If you want a more sophisticated custom-architecture feature, stochastic depth would be especially interesting because it fits the residual structure mechanically, rather than just inserting generic dropout everywhere.

gpt-5-6-thinking

i've read about stochastic depth, how might that affect things too? should that be added? how might I go about doing so?

Stochastic depth is a particularly good fit for the residual network you're building because it regularizes at the **block level**, not just at the neuron level. The main question is how aggressively to apply it and where.

Yes. For a **custom CIFAR residual network**, stochastic depth is arguably a more architecturally natural regularizer than putting ordinary dropout throughout the convolutional blocks.

The key distinction is:

- **Dropout:** randomly removes individual activations.
- **Stochastic depth:** randomly removes an **entire residual transformation**.

For a residual block,

$$y = x + F(x)$$

stochastic depth changes training so that sometimes the block behaves like

$$y = x$$

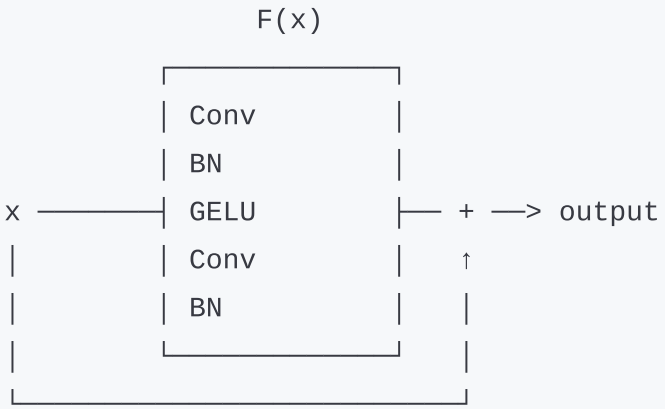
instead of

$$y = x + F(x)$$

The skip connection remains intact, so information and gradients can still flow through the network.

What it actually does during training

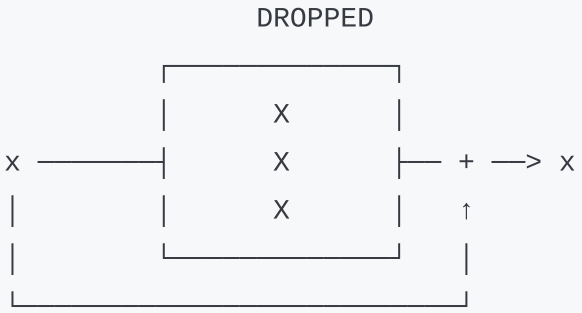
Suppose you have:



Normally:

$$y = x + F(x)$$

With stochastic depth, on some training passes the entire `F(x)` branch disappears:



so:

$$y = x$$

On another sample/pass, the block is active again:

$$y = x + F(x)$$

Thus the network can't become completely dependent on every residual block.

Why can this improve generalization?

There are several effects happening simultaneously.

1. Your network effectively has varying depth

Suppose your custom network contains 8 residual blocks.

Without stochastic depth:

```
Every training example:
Block 1
  ↓
Block 2
  ↓
Block 3
  ↓
...
Block 8
```

With stochastic depth:

```
Image A:
1 → 2 → X → 4 → 5 → X → 7 → 8

Image B:
1 → X → 3 → 4 → X → 6 → 7 → 8

Image C:
1 → 2 → 3 → X → 5 → 6 → X → 8
```

You're effectively training an ensemble of networks with slightly different depths.

At inference time, all blocks are available.

That discourages pathological dependencies like:

> *"Block 6 only works if Block 5 produced exactly this feature."*

Instead, blocks have to operate more robustly.

---

2. It regularizes feature transformations rather than individual pixels

That's particularly nice for CNNs.

Ordinary dropout might randomly remove:

$$x_{c,h,w}$$

values inside a feature map.

Stochastic depth instead says:

> *"For this example, don't use this entire learned transformation."*

That's much better aligned with the semantics of a residual network.

---

3. It provides extremely clean gradient paths

This is one of my favorite properties.

Consider

$$y = x + F(x)$$

The derivative is

$$\frac{\partial y}{\partial x} = 1 + \frac{\partial F}{\partial x}$$

If the residual branch gets dropped:

$$y = x$$

then

$$\frac{\partial y}{\partial x} = 1$$

So gradients can pass directly backward through that block.

That's why stochastic depth works particularly well with residual architectures: **the skip connection gives you a safe bypass.**

How I'd add it to your block

Torchvision already provides this:

```
from torchvision.ops import StochasticDepth
```

Its current API takes a drop probability and either `"row"` or `"batch"` mode. `"row"` independently applies the stochastic-depth decision to samples in the batch, whereas `"batch"` applies it to the entire batch.

For your network I'd use:

```
StochasticDepth(p, mode="row")
```

Then modify your block like this:

```
import torch
import torch.nn as nn
from torchvision.ops import StochasticDepth


class Block(nn.Module):
    def __init__(
        self,
        in_channels,
        out_channels,
        stride=1,
        drop_path=0.0
    ):
        super().__init__()

        self.conv1 = nn.Conv2d(
            in_channels,
            out_channels,
            kernel_size=3,
            stride=stride,
            padding=1,
            bias=False
        )

        self.bn1 = nn.BatchNorm2d(out_channels)
        self.act = nn.GELU()

        self.conv2 = nn.Conv2d(
```

```
            out_channels,
            out_channels,
            kernel_size=3,
            padding=1,
            bias=False
        )

        self.bn2 = nn.BatchNorm2d(out_channels)

        self.drop_path = StochasticDepth(
            drop_path,
            mode="row"
        )

        if stride != 1 or in_channels != out_channels:
            self.skip = nn.Sequential(
                nn.Conv2d(
                    in_channels,
                    out_channels,
                    kernel_size=1,
                    stride=stride,
                    bias=False
                ),
                nn.BatchNorm2d(out_channels)
            )
        else:
            self.skip = nn.Identity()

    def forward(self, x):

        identity = self.skip(x)

        out = self.conv1(x)
        out = self.bn1(out)
        out = self.act(out)

        out = self.conv2(out)
        out = self.bn2(out)

        # Drop the RESIDUAL BRANCH
        out = self.drop_path(out)
```

```
        out = identity + out
        out = self.act(out)

        return out
```

The placement is important:

```
Conv
 ↓
BN
 ↓
GELU
 ↓
Conv
 ↓
BN
 ↓
STOCHASTIC DEPTH   ← here
 ↓
+ skip connection
 ↓
GELU
```

You generally **do not** want:

```
out = self.drop_path(identity + out)
```

because then you're potentially dropping the skip connection too.

The whole point is:

$$\boxed{x + \mathrm{DropPath}(F(x))}$$

not

$$\mathrm{DropPath}(x + F(x)).$$

Torchvision's implementation is specifically intended for dropping residual branches this way.

Don't give every block the same probability

Here's where stochastic depth gets more interesting.

Usually, shallow blocks have a very low chance of being dropped and deeper blocks have a higher chance.

For example, with 8 blocks and maximum drop probability

$$p_{\max} = 0.10$$

you could use:

```
Block 1    0.000
Block 2    0.014
Block 3    0.029
Block 4    0.043
Block 5    0.057
Block 6    0.071
Block 7    0.086
Block 8    0.100
```

Why?

Early layers learn fundamental features:

```
edges
 ↓
colors
 ↓
textures
 ↓
simple shapes
```

Deep layers learn increasingly abstract transformations.

Dropping an early block is therefore usually more disruptive than dropping a late one.

So you commonly use a linear schedule:

$$p_i = p_{\max} \frac{i}{N-1}.$$

In Python:

```
drop_rates = torch.linspace(
    0,
    0.1,
    steps=8
).tolist()
```

Applied to your architecture

Earlier we had roughly:

```
Stage 1: 2 blocks @ 64
Stage 2: 2 blocks @ 128
Stage 3: 2 blocks @ 256
Stage 4: 2 blocks @ 512
```

I'd construct it something like this:

```
class CIFARNet(nn.Module):
    def __init__(self, num_classes=10, max_drop_path=0.1):
        super().__init__()

        rates = torch.linspace(
            0,
            max_drop_path,
            8
        ).tolist()

        self.stem = nn.Sequential(
            nn.Conv2d(
                3, 64, 3,
                padding=1,
                bias=False
            ),
            nn.BatchNorm2d(64),
            nn.GELU()
        )

        self.stage1 = nn.Sequential(
            Block(64, 64, drop_path=rates[0]),
            Block(64, 64, drop_path=rates[1])
```

```
        )

        self.stage2 = nn.Sequential(
            Block(64, 128, stride=2, drop_path=rates[2]),
            Block(128, 128, drop_path=rates[3])
        )

        self.stage3 = nn.Sequential(
            Block(128, 256, stride=2, drop_path=rates[4]),
            Block(256, 256, drop_path=rates[5])
        )

        self.stage4 = nn.Sequential(
            Block(256, 512, stride=2, drop_path=rates[6]),
            Block(512, 512, drop_path=rates[7])
        )

        self.pool = nn.AdaptiveAvgPool2d(1)

        self.dropout = nn.Dropout(0.2)

        self.fc = nn.Linear(
            512,
            num_classes
        )

    def forward(self, x):

        x = self.stem(x)

        x = self.stage1(x)
        x = self.stage2(x)
        x = self.stage3(x)
        x = self.stage4(x)

        x = self.pool(x)
        x = torch.flatten(x, 1)

        x = self.dropout(x)

        return self.fc(x)
```

Now you've got two **different kinds of regularization**:

```
Residual feature extractor
        |
        | ← stochastic depth
        |    regularizes BLOCKS
        ↓
AdaptiveAvgPool
        ↓
[batch, 512]
        |
        | ← dropout
        |    regularizes FEATURES
        ↓
Classifier
```

That's a defensible architecture.

How strong should it be?

This is where I'd be conservative.

Your network only has around 8 residual blocks. It's not a 100- or 200-layer monster.

I would test:

Experiment	Max stochastic-depth probability
Baseline	`0.00`
Light	`0.05`
Moderate	`0.10`
Strong	`0.20`

I would expect **0.05-0.10** to be the most interesting range for this architecture.

`0.5` would be pretty extreme for only eight blocks.

If your deepest block has

```
p = 0.1
```

then it survives with probability

$$1 - p = 0.9.$$

That's enough randomness to regularize it without making optimization chaotic.

One important implementation detail

Stochastic depth automatically switches behavior when you do:

```
model.train()
```

versus:

```
model.eval()
```

During training:

```
some residual branches disappear
```

During evaluation:

```
all residual branches are active
```

The operator handles the appropriate scaling so that the activation magnitude remains consistent between training and inference.

Conceptually, if the branch survives with probability

$$q = 1 - p,$$

the surviving residual branch is scaled so its expected contribution remains approximately unchanged:

$$E\left[\frac{m}{1-p}F(x)\right] = F(x),$$

where

$$m \sim \text{Bernoulli}(1-p).$$

That's analogous to the scaling used by normal dropout.

Would I actually add it?

Yes, but as a controlled architectural experiment.

For your assignment, you've now got several meaningful design decisions:

```
32×32 CIFAR input
        ↓
3×3 stride-1 stem
        ↓
No initial max pooling
        ↓
Residual blocks
        ↓
GELU
        ↓
Progressive stochastic depth
        ↓
Global average pooling
        ↓
Dropout
        ↓
Classifier
```

There's a coherent reason for every piece.

But I wouldn't immediately turn every regularizer up:

```
heavy augmentation
+ huge weight decay
+ dropout 0.5
+ stochastic depth 0.3
```

That can cause **underfitting**.

Instead I'd start with something like:

```
weight_decay = 5e-4
dropout = 0.2
```

```
max_stochastic_depth = 0.1
```

plus normal CIFAR augmentation such as random crop and horizontal flip.

Then run the most informative ablation:

Experiment	Dropout	Stoch. depth
Baseline	0	0
A	0.2	0
B	0	0.1
C	0.2	0.1

That tells you something much more useful than merely reporting that your final architecture got a particular accuracy: **you'll be able to show which regularization mechanism actually contributed to generalization.**

For an architecture assignment, that analysis is probably as valuable as the architecture itself.

gpt-5-6-thinking